👷🛠️Course Project🛠️💪

Use this (depending of what group you are in):
.NET: C# (required), Blazor or MVC (you choose); of course, you use JavaScript or TypeScript as needed
or
JavaScript: JavaScript or TypeScript (you choose), React, Express (you can choose other framework instead Express if you want)

Use CSS framework (Bootstrap is recommended, but you can use _any_).

```You can use any database and any other libraries, components or even frameworks (but not replace specified above).```

There are no limitations in the area of architecture or used services. For example, you are not required to have a separate full-featured application server separated from Web server, or have a lot of things on the client, or use microservices, etc.; you may approach this whatever way you want.

It's recommended to use _the simplest and the safest_ approach to the persistence, namely relational databse, e.g. PostgreSQL, MySQL, SQL Server, anything, but again you don't have to, you may try to use MongoDB or something (for this task it's _definitely_ not the best idea, meh...). Again, you may replace Bootstrap with any CSS framework and/or UI library you like.

React is required for JavaScript group; it can be used by .NET group as well.

You have to implement a **Web application for inventory management** (office equipment, books in library, documents in HR department, etc.). Users define arbitrary "**inventories**" (the set of fields with a title and description, etc.), and other users fill inventories with "**items**" (specific) using these templates (e.g., enter or select values in the fields). Two the most important features are custom inventory numbers (IDs) and custom fields.

E.g., I create an inventory with a string-valued field "Model" and a number-valued field "Price". Other users add their laptops to this inventory.

If your app will have N buttons (view/edit/delete in _each_ item), your result will be graded -20%. Use toolbars, or animated "appearing" context actions, etc.

This is forbidden:

```
┌──────────┬──────────────────┬──────┬──────────────────┐
│ ID       │ Equipment        │ .....│                  │
├──────────┼──────────────────┼──────┼──────────────────┤
│ XD_6332  │ Personal laptop  │ .....│ [Edit][Delete]   │
├──────────┼──────────────────┼──────┼──────────────────┤
│ XN_23FA  │ Fax machine      │ .....│ [Edit][Delete]   │
├──────────┼──────────────────┼──────┼──────────────────┤
│          │                  ...    │                  │
├──────────┼──────────────────┼──────┼──────────────────┤
│ YN_544C  │ Workstation      │ .....│ [Edit][Delete]   │
└──────────┴──────────────────┴──────┴──────────────────┘
```
```

This is OK:

```

[Delete]

| ☐ | ID | Equipment | Year |
|---|---|---|---|
| ☐ | XD_6332 | Personal laptop | 2025 |
| ☑ | XN_23FA | Fax machine | 2023 |
| ☐ | YN_544C | Workstation | 2027 |

```

> You have to use table representation for items and inventories, no ~~gallery~~ or ~~tiles~~.

Every page provides access to full-text search via the top header.

Inventories, as well as items, should be displayed in table views by default. You may add other display options, but do not replace the tables.

Non-authenticated users cannot create inventories, leave comments or likes, or add items. However, they can use the search function and view all inventories and items in read-only mode.

The admin page allows user management, including: viewing, blocking, unblocking, deleting users, adding users to the admin role, and removing them from it.

> Important: AN ADMIN CAN REMOVE ADMIN ACCESS FROM THEMSELVES.

Admins have full access and can view all pages as if they were the creator. For example, an admin can open another user's inventory and add fields or edit items—effectively acting as the owner of every inventory.

Only the creators (and admins) can edit their inventories (e.g., add/edit/delete fields, manage access, etc.). Creators define which users have write access (add/edit/delete) to their inventory. Creators can also mark an inventory as public, granting write access to all authenticated users.

Items in an inventory can be modified by the creator (and admins), and any users with write access (or by all authenticated users if the inventory is public).

All items are viewable by everyone, regardless of authentication status.

Users can register and authenticate via social netowrk (at least two; Google and Facebook are very good options).

Every user has their own personal page, where they can manage two sortable/filterable tables:

1. A table of inventories they own (with options to create new, delete, or edit).
2. A table of inventories they have write access to.

Each inventory in the table functions as a link to its dedicated page, which contains several tabs:

- Table of items in the inventory (with links to individual item pages).
- Discusison section.
- General settings — title, description, etc.
- **Custom inventory numbers.**
- Access settings — either all authenticated users (for public inventories) or specific users (for non-public ones).
- **Editable set of fields for all items in the inventory.**
- Inventory statistics/aggregation, such as the number of items, averages and ranges for numeric fields, most frequently used values for string fields, etc. The owner doesn't edit anything on this tab.

For users with write access (but not ownership), only the 1st and 2nd tabs are accessible in edit mode (items).

The inventory author and users with write access can:

- Click any item to open it in edit mode.
- Delete existing items.
- Add new items.

Users without write access can only view items; they cannot add, edit, or delete them.

> Access is managed by maintaining a list of users, with the ability to remove users from the list and add new ones by typing a username or email address (autocomplete is required both by name and e-mail). The list is sortable, with a user-switchable sort mode (by name or by email).

> Inventory page supports **auto-save** capability (it's not required for items). Don't try to save every user keystroke; instead, track changes and save every 7–10 seconds. Beware of optimistic locking—each save operation either updates a version in the database and returns it for the next save, or fails.

Inventory settings include the following:

- Title.
- Description (supports Markdown formatting).
- Category — a single value selected from a predefined list, e.g., Equipment, Furniture, Book, or Other (Note: new values are added directly in the database; no UI is required for this).
- Optional image/illustration — uploaded by the user to the cloud.
- Tags — users can enter multiple tags. Autocomplettion for tags is required: when the user starts typing, a dropdown should appear with existing tags from the database that start with the entered text.

Each inventory can be marked as public (allowing any authenticated user to add items), or the creator can manually select a list of registered users who are granted access.

The <u>first "killer-feature" is that an inventory allows the specification of custom fields for its items</u>.

In addition to the globally unique internal ID (not displayed in the UI), each inventory can define its own custom ID format, which generates a unique ID specific to that inventory. This format is customizable.

- Custom IDs are unique within the scope of a single inventory. Items in different inventories may have the same custom ID.

Uniqueness is enforced at the database level—it's not possible to insert or update an item with a duplicate ID within the same inventory.

- The custom ID is not the primary key (unique for the whole table and permanent) in the **`[items]`** table, but is instead managed by a separate composite database index (**`inventory_id`** plus **`custom_id`**).
- Custom IDs are editable. On the item form, the ID is edited as a single string using one input field, with format validation applied.

When an item is created, the custom ID is generated by the system and saved to the database. If the custom ID format is later changed for the inventory, existing item IDs remain unchanged; however, during item editing, the new ID format rules are enforced.

The Custom ID tab includes a real-time example preview of the resulting ID.

The following ID parts (elements) are supported:

- Fixed text (with full Unicode support),
- 20-bit random number,
- 32-bit random number,
- 6-digit random number,
- 9-digit random number,
- GUID,
- Date/time (at the moment of item creation),
- Sequence (value equal to the largest existing sequence number +1 at the moment of item creation).

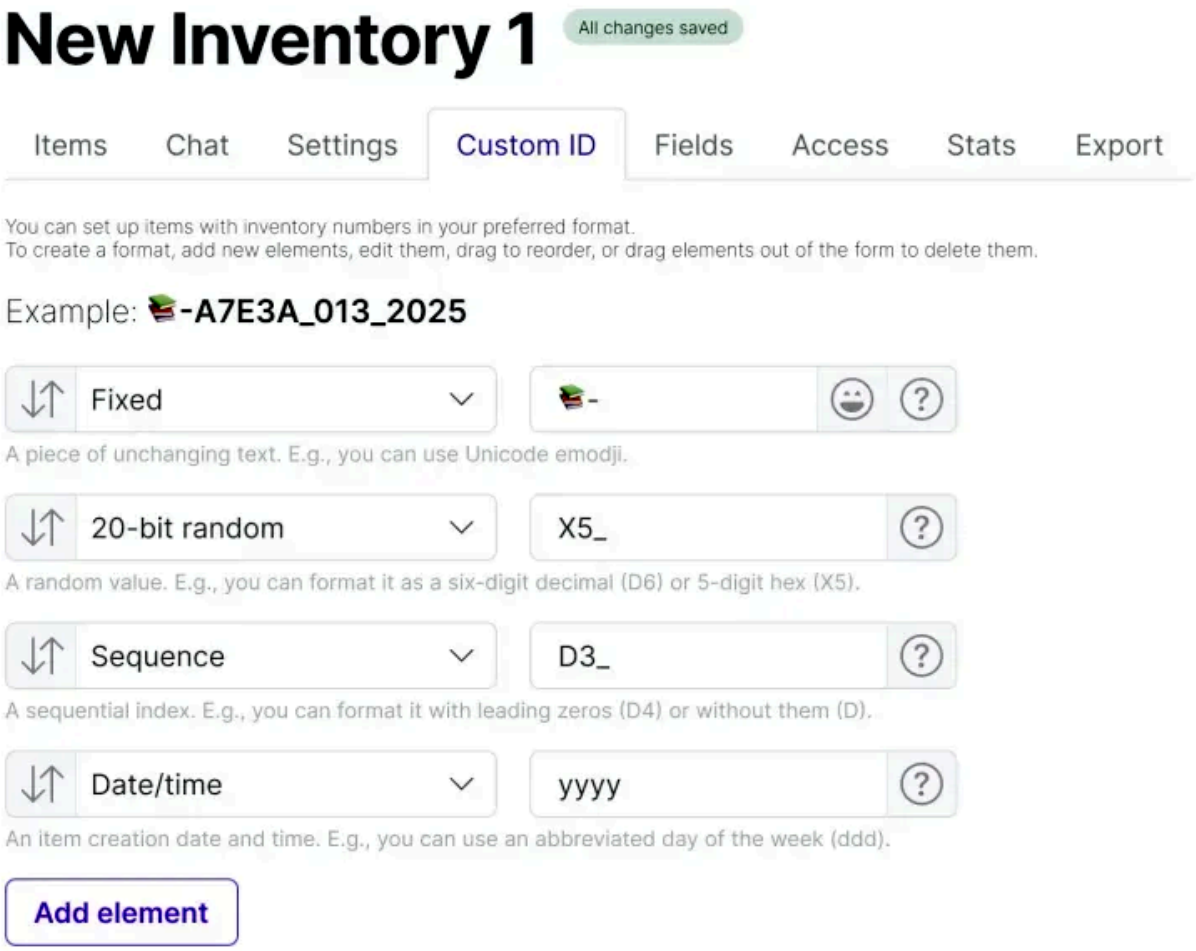
Users can:

- Reorder ID elements via drag-and-drop.
- Remove elements by dragging them outside the form.
- Add new elements, with a recommended upper limit of at least 10 elements.
- Change element formatting rules, e.g., format numbers with leading zeros.

The form should include detailed help, using popovers, to guide users through formatting options.

Please note that the illustration below is not a complete page. You also need site navigation, account-related features—such as login/logout, language selection, and theme switching, and more, search functionality, etc.

Of course, conflicts can occur during item submission when an item is created or edited (e.g., multiple users add items in parallel, a randomly generated value is duplicated, etc.). In such cases, the database should reject the items, and the user will need to manually edit the custom ID value.



The <u>second "killer-feature" is that an inventory allows the specification of custom fields for its items</u>.

There are also fixed fields that are always present. These fields are not visible in the UI for field customization but are displayed on every item form. For example, automatically generated fields like `**created_by**`, `**created_at**`, or `**custom_id**`—the latter being editable despite its automatic generation.

In addition to the fixed fields, it's possible to add custom fields, with the following limitations:

- Up to 3 single-line text fields.
- Up to 3 multi-line text fields.
- Up to 3 numeric fields.
- Up to 3 document/image fields (entered as a link).
- Up to 3 true/false fields (displayed as checkboxes).

Each custom field includes:

- Title.
- Description (displayed as a tooltip or hint in forms).
- A boolean flag indicating whether the field should be displayed in the item table view (on the inventory's tab).

> Fields can be reordered via drag-and-drop.

For example, suppose I want to create an inventory for books in a library. I might add the following fields:

- Single-line field: "Title"
- Single-line field: "Authors"
- Numeric field: "Year"
- Multi-line field: "Annotation"

The main page of the app contains:

- A table of the latest inventories, showing the name, description (or image), and creator.
- A table of the top 5 most popular inventories, based on the number of items.
- A tag cloud — when a user clicks a tag, a list of related inventories is displayed (you should reuse the standard “search results” page layout for that)

Each inventory page includes a discussion tab. Posts are linear—new posts are always added to the end; inserting posts between existing ones is not allowed. Posts should be updated automatically—when someone adds a post, it should appear for all users viewing the page within 2–5 seconds (real-time or near real-time updates). Every post displays Markdown text, user name (which works as a link to the user's personal page), and a date/time.

Each item has a "like" feature (a user can give one like per item, and no more than one like from the same user is allowed).

> You need to implement _optimistic_ locking for inventories (which can be modified simultaneously by the creator and admins) and for items (which are modifiable by multiple users with write access at the same time).

The application should support two UI languages: English and one additional language (e.g., Polish, Spanish, Uzbek, Georgian, etc.). The user selects the language, and their choice is saved. *Only the UI is translated* — user-generated content such as inventories or items is not translated.

The application should also support two visual themes: light and dark. The user selects the theme, and their choice is saved.

Requirements:

- Use a CSS framework (e.g., Bootstrap — or any other framework and set of UI controls you prefer).
- Support responsive design for various screen sizes and resolutions, including mobile phones.
- Use an ORM (e.g., Sequelize, Prisma, TypeORM, Entity Framework — any is acceptable).
- Use a full-text search engine, either through an external library or native database features.

DON'Ts:

- Don’t perform full database scans using raw SELECT * queries.
- Don’t upload images to your web server or database.

- Don’t execute database queries inside loops.
- Don't add buttons in the table rows.

> Is it possible to use the X library? `Yes`, yes to all — remember my choice.

Optional Requirements

(for a separate grade—only if all core requirements are fully implemented)

- Display document previews for document/image links in items (e.g., JPG and PDF).
- Implement form authentication with email confirmation as an alternative to social network authentication.
- Add additional “tuning” options to the fields, such as: length limits and regex validators for string fields, value ranges for numeric fields, etc.
- Add a field type “one from the given list”, with the ability to define the list of options (for example, the inventory creator defines a "Type" field with options like "Desktop" / "Laptop" / "Table".
- Allow an arbitrary number of fields of any type (not limited to 0, 1, 2, or 3—no upper limit).
- Inventory export to CSV/Excel.

IMPORTANT NOTE

Do NOT copy code from code dumps or old repositories, reqs are changing in small details. Don't hope that AI will undestand all the requirements perfectly. **Anyway, it's much better to do less, but fully understand what you write.** I'm dead serious; we will ask you to modify your code on the fly, add or change functionality, as well as ask technical questions. We want not only to check your ability to navigate and work with your own code, but to undestand _why_ every line is written. This is more important than the number of features you implement.

Your supervisor has seen many similar projects and is likely familiar with most publicly available examples online. Do not copy. **Use libraries as much as possible**—but don't copy-paste code.

> Use ready-made components, libraries, and controls.

For example, use a component to render Markdown, use an image uploader with drag-and-drop support, use a tag input control, use a tag cloud renderer, etc.

> The less custom code your app contains, the better.

And the most important: ***start by deploying a static "Hello, world" page and always keep a deployable version ready!***

And even more important than that: ***defend your project—even if you’ve only completed a small part of it***.

Good luck!

Q: "I did very few tasks, is it OK?", etc.

A: From the very beginning I mentioned that it's OK if you don't finish some of the tasks. It's possible to implement less than 100% tasks and less that 100% of project requirements.

Some did more tasks, but some did few task better. Some will do project better. It's not just a "number of tasks". As Marcus Aurelius said, "Do less, but better."

Also, here is another good point of view. Do not worry, but try your best. As one Frenchman said, "*Do what you ought, come what may*."